Logistic Regression

Aim: To build and evaluate a Logistic Regression model for irrigation prediction and analyze probability outputs.

```
import pandas as pd
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
path="/content/drive/MyDrive/ML datasets/smart_irrigation_dataset1.csv"
df=pd.read_csv(path)
df.head(5)
```

	Temperature	Humidity	Soil_Moisture	Rainfall	Irrigation
0	32.57	74.89	39.76	8.73	0
1	39.66	20.05	62.95	2.63	0
2	27.28	80.13	16.00	0.32	1
3	34.55	52.32	45.56	14.21	0
4	24.92	50.13	26.45	8.21	0

Next steps: Generate code with df New interactive sheet

```
df.describe()
```

	Temperature	Humidity	Soil_Moisture	Rainfall	Irrigation
count	1500.000000	1500.000000	1500.000000	1500.000000	1500.000000
mean	27.561700	60.191187	42.398653	10.141427	0.078000
std	7.237384	22.977413	21.429685	5.836865	0.268261
min	15.010000	20.050000	5.020000	0.010000	0.000000
25%	21.547500	40.452500	24.170000	5.047500	0.000000
50%	27.395000	60.695000	41.845000	10.230000	0.000000
75%	33.992500	79.645000	60.922500	15.315000	0.000000
max	39.980000	99.930000	79.980000	19.990000	1.000000

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1500 entries, 0 to 1499
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Temperature   1500 non-null   float64
 1   Humidity      1500 non-null   float64
 2   Soil_Moisture 1500 non-null   float64
 3   Rainfall      1500 non-null   float64
 4   Irrigation    1500 non-null   int64
dtypes: float64(4), int64(1)
memory usage: 58.7 KB
```

```
df.shape
```

```
(1500, 5)
```

```
df.columns
```

```
Index(['Temperature', 'Humidity', 'Soil_Moisture', 'Rainfall', 'Irrigation'], dtype='object')
```

```
df.isnull().sum()
df.isnull().any().any()
df[df.isnull().any(axis=1)]
```

Temperature	Humidity	Soil_Moisture	Rainfall	Irrigation

```
before_rows=df.shape[0]
df=df.dropna()
after_rows=df.shape[0]

print("Before rows:", before_rows)
print("After rows:", after_rows)
print("Dropped rows:", before_rows - after_rows)
```

```
Before rows: 1500
After rows: 1500
Dropped rows: 0
```

```
clean_data = df.copy()
clean_data.columns
clean_data.shape
```

```
(1500, 5)
```

```
X = clean_data.iloc[:,:-1]
y = clean_data.iloc[:,-1]

print(X)
print(y)
```

```
     Temperature  Humidity  Soil_Moisture  Rainfall
0          32.57     74.89          39.76      8.73
1          39.66     20.05          62.95      2.63
2          27.28     80.13          16.00      0.32
3          34.55     52.32          45.56     14.21
4          24.92     50.13          26.45      8.21
...          ...       ...            ...       ...
1495       33.50     69.60          35.67      6.57
1496       36.66     50.69          51.16      8.87
1497       38.91     35.98          70.58     17.06
1498       39.15     75.90          34.62      2.59
1499       23.40     43.66          57.73     10.79

[1500 rows x 4 columns]
0      0
1      0
2      1
3      0
4      0
      ..
1495   0
1496   0
1497   0
1498   0
1499   0
Name: Irrigation, Length: 1500, dtype: int64
```

```
from sklearn.model_selection import train_test_split

X_train,X_test,y_train,y_test = train_test_split(
    X, y, train_size=0.7, test_size=0.3, random_state=42
)
```

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
lr_model = LogisticRegression(max_iter=1000)
lr_model.fit(X_train, y_train)
```

```
LogisticRegression
LogisticRegression(max_iter=1000)
```

```
y_pred = lr_model.predict(X_test)
y_pred
```

```
array([0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0,
       0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1,
       0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0,
       0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0,
       1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0,
       0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       1, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
print("\n--- Evaluation Metrics ---")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, average='weighted'))
print("Recall:", recall_score(y_test, y_pred, average='weighted'))
print("F1 Score:", f1_score(y_test, y_pred, average='weighted'))
```

```
--- Evaluation Metrics ---
Accuracy: 0.9511111111111111
Precision: 0.9500709703060239
Recall: 0.9511111111111111
F1 Score: 0.9505438060182586
```

```
from sklearn.metrics import confusion_matrix, classification_report

print(confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
[[400  10]
 [ 12  28]]
Accuracy: 0.9511111111111111
              precision    recall  f1-score   support

           0       0.97      0.98      0.97       410
           1       0.74      0.70      0.72        40

    accuracy                           0.95       450
   macro avg       0.85      0.84      0.85       450
weighted avg       0.95      0.95      0.95       450
```

```
probabilities = lr_model.predict_proba(X_test[:10])
print("Prediction Probabilities:\n", probabilities)
```

```
Prediction Probabilities:
 [[9.93888773e-01 6.11122684e-03]
 [9.97817112e-01 2.18288796e-03]
 [9.99999729e-01 2.70517437e-07]
 [9.98674147e-01 1.32585300e-03]
 [9.99995500e-01 4.50045885e-06]
```

```
[3.75744669e-01 6.24255331e-01]
[9.99940313e-01 5.96873620e-05]
[9.99999916e-01 8.35518055e-08]
[9.73835332e-01 2.61646677e-02]
[9.69300063e-01 3.06999371e-02]]
```